Swarm Logistics & Evacuation Dynamics

Guida Pratica allo Sviluppo del Codice — v2

Corso di Artificial & Swarm Intelligence - A.A. 2025/2026

Massimiliano Cassia

Griglia	Agenti	Batteria	Tick Max
25x25 - 2D	**5 (eterogenei)**	**500 u. (-1/mossa)**	**500 - 750**

Include integrazione con i file del professore (visualize_environment.py e README)

0 File Forniti dal Professore

Analisi di A.json, B.json, visualize_environment.py e README — da leggere PRIMA di scrivere codice

Prima di aprire qualsiasi editor, esegui visualize_environment.py su entrambe le istanze. Capire visivamente la topologia vale piu' di qualsiasi lettura del JSON grezzo: vedrai dove sono i 4 magazzini, gli ENTRANCE, gli EXIT e dove il professore ha disperso i 10 oggetti.

0.1 Struttura dei file JSON (A.json e B.json)

Campo	Tipo	Contenuto e uso nel codice
metadata	dict	grid_size (25), num_warehouses (4), num_objects (10). Usa metadata['grid_size'] invece di hardcodare 25.
grid	list[list[int]] 25x25	Matrice r,c con valori 0/1/2/3/4. E' la mappa statica: muri, magazzini, entrance, exit. NON contiene la posizione degli oggetti.
warehouses	list[dict]	Lista di 4 dizionari. Ogni dict ha: 'entrance': [r,c], 'exit': [r,c], 'side': 'top'/'bottom'/'left'/'right'. La 'side' indica il lato del magazzino e serve per sapere la direzione dei sensi unici.
objects	list[[r,c]]	GROUND TRUTH del simulatore. NON passarla agli agenti. Gli agenti devono scoprire la posizione solo tramite sensore visivo durante l'esplorazione.

0.2 Il campo 'warehouses' e i sensi unici — dettaglio critico

Il README del professore scrive is_walkable() senza implementare i sensi unici, delegando a te. NON copiare quella funzione. Il campo 'side' di ogni magazzino e' la chiave per implementare correttamente la direzionalita':

side	ENTRANCE: direzione valida	EXIT: direzione valida	Logica in is_walkable()
top	Agente viene da NORD (r diminuisce)	Agente esce verso SUD (r aumenta)	ENTRANCE: agent_r < entrance_r. EXIT: agent_r > exit_r.
bottom	Agente viene da SUD (r aumenta)	Agente esce verso NORD (r diminuisce)	ENTRANCE: agent_r > entrance_r. EXIT: agent_r < exit_r.
left	Agente viene da OVEST (c diminuisce)	Agente esce verso EST (c aumenta)	ENTRANCE: agent_c < entrance_c. EXIT: agent_c > exit_c.
right	Agente viene da EST (c aumenta)	Agente esce verso OVEST (c diminuisce)	ENTRANCE: agent_c > entrance_c. EXIT: agent_c < exit_c.

0.3 visualize_environment.py — come usarlo e cosa imparare

PRIMA COSA DA FARE: genera le immagini delle due mappe

```
cd swarm_logistics_project
python tools/visualize_environment.py data/A.json data/A.png
python tools/visualize_environment.py data/B.json data/B.png
```

Apri le PNG e studia: dove sono i 4 magazzini? Quali corridoi sono a 1 cella? Dove si trovano gli oggetti?
Questa analisi visiva guidera' le tue scelte implementative (es. dove si formeranno i colli di bottiglia).

Due dettagli tecnici da ereditare nel tuo renderer.py:

- ax.invert_yaxis(): obbligatorio. Conferma che riga 0 e' in ALTO. Il tuo renderer deve replicarlo, altrimenti la visualizzazione live sara' capovolta rispetto al PNG del professore.
- Le coordinate degli oggetti nel visualizzatore sono (obj_r, obj_c) con Circle((obj_c, obj_r), ...) — notare lo swap per matplotlib (x=colonna, y=riga). Tienilo a mente quando disegni gli agenti nel renderer.

0.4 Gli oggetti sono ground truth — implicazione architetturale

ATTENZIONE: questo vincolo ha impatto diretto su environment.py

env['objects'] e' la lista ground truth del simulatore. Va caricata in Environment come attributo privato.
Gli agenti NON devono avere accesso diretto a questo attributo.
La scoperta avviene SOLO quando il sensore visivo dell'agente interseca la cella dell'oggetto durante l'esplorazione.
Quando un oggetto viene consegnato: rimuovilo da objects_truth (es. objects_truth.discard((r,c))).
Per il debug usa objects_truth liberamente; in produzione non passarla a nessun agente.

```
# In environment.py — separazione corretta
class Environment:
    def load(self, path):
        with open(path) as f:
            data = json.load(f)
        self.grid        = data['grid']             # mappa statica
        self.warehouses  = data['warehouses']       # entrance, exit, side
        self._objects_truth = set(map(tuple, data['objects']))  # PRIVATO
        self.n           = data['metadata']['grid_size']
        self.stigma_map  = [[0.0]*self.n for _ in range(self.n)]
        self.occupancy   = set()
        self.traffic_log = {}   # {(r,c): count} per heatmap
        self.delivered   = 0

    def is_object_at(self, r, c):  # chiamato SOLO dal sensore, mai dagli agenti
        return (r, c) in self._objects_truth

    def deliver_object(self, r, c):
        self._objects_truth.discard((r, c))
        self.delivered += 1
```

1 Prerequisiti Architetturali

Da applicare dal primo rigo di codice — non sono opzionali

PREREQUISITO 1 — Sistema di Coordinate (r, c) [Checklist #13]

Usa SEMPRE (riga, colonna) o (r, c). Mai (x, y).
Confermato dal README del professore: 'Coordinate sempre in formato [riga, colonna], con origine in alto a sinistra [0,0]'.
In matplotlib: x=colonna, y=riga (sono invertiti). Il visualizzatore del prof usa Circle((obj_c, obj_r)) — copia questo pattern.

PREREQUISITO 2 — Seed Fisso [Checklist #15]

random.seed(42) e numpy.random.seed(42) come prime righe di config.py.
Senza seed fisso i bug cambiano ad ogni run e diventano impossibili da replicare.
Usa seed variabili SOLO per le run statistiche finali (N=5).

PREREQUISITO 3 — Flag GUI e config.py [Checklist #16]

Crea src/config.py come primo file del progetto. Ogni costante numerica va qui.
Includi GUI=True/False. Con GUI=False le run statistiche passano da minuti a frazioni di secondo.
Senza config.py centralizzato, cambiare VISION_RADIUS per la configurazione C1 richiede cercare in 5 file.

Contenuto completo di src/config.py

```
import random, numpy as np

# -- Riproducibilita' ----------------------------------------
SEED = 42
random.seed(SEED)
np.random.seed(SEED)

# -- Ambiente ------------------------------------------------
GRID_SIZE         = 25   # da metadata['grid_size'], non hardcodare
EMPTY, WALL       = 0, 1
WAREHOUSE         = 2
ENTRANCE          = 3
EXIT              = 4
NUM_OBJECTS       = 10
MAX_TICKS         = 750

# -- Agenti --------------------------------------------------
NUM_AGENTS        = 5
BATTERY_INITIAL   = 500
ENERGY_MARGIN     = 1.20   # Worker 3: +20% di sicurezza

# -- Sensori e Comunicazione ---------------------------------
VISION_RADIUS     = 3      # C1: cambia questo (1 vs 3)
```

```
COMM_RADIUS       = 2
DEPLOY_RADIUS     = 1

# -- Stigmergia ----------------------------------------------
CONGESTION_MALUS  = 10
EVAPORATION_RATE  = 0.05
STIGMA_ON         = True   # C2: cambia questo (True vs False)

# -- Scout 1 -------------------------------------------------
STRESS_MAX        = 15
STRESS_RANDOM_STEPS = 4

# -- Visualizzazione -----------------------------------------
GUI               = True   # False per run statistiche
```

2 Struttura del Progetto

Organizzazione dei file e responsabilita' di ogni modulo

```
swarm_logistics_project/
|
+-- data/
|   +-- A.json  B.json          # mappe del professore (NON modificare)
|   +-- A.png   B.png           # genera con visualize_environment.py
|
+-- src/                        # motore logico
|   +-- config.py               # PREREQUISITO: tutte le costanti
|   +-- main.py                 # entry point per 1 singola run visiva
|   +-- environment.py          # griglia, ground truth oggetti, stigmergia,
|   |                           # spawn queue, enforcement sensi unici
|   +-- communication.py        # struttura messaggi FIPA-ACL (utility pura)
|   +-- pathfinding.py          # A* + path caching + real_distance()
|   +-- sensors.py              # raycasting + detect_agents
|   |
|   +-- agents/
|   |   +-- __init__.py
|   |   +-- base_agent.py       # classe madre: stati, batteria, merge_knowledge
|   |   +-- scouts.py           # Scout1 (Wall-Follower), Scout2 (Stigmergia)
|   |   +-- workers.py          # Worker1 (Greedy), Worker2 (Densita'), Worker3
|   |
|   +-- utils/
|       +-- logger.py           # eventi in RAM, dump JSON unico a fine run
|       +-- renderer.py         # matplotlib isolato, eredita da visualize_env.py
|
+-- experiments/
|   +-- run_baseline.py         # N=5 run B0 (Random Walk)
|   +-- run_configs.py          # N=5 run per C1, C2, C3
|   +-- analyze_results.py      # heatmap, grafici, statistiche comparative
|
+-- outputs/                    # aggiungere a .gitignore
|   +-- logs/  heatmaps/  plots/
|
+-- tools/
|   +-- visualize_environment.py  # script originale del professore
|
+-- requirements.txt  README.md
```

Responsabilita' per modulo

Modulo	Contiene	NON contiene
config.py	Tutte le costanti, seed, flag GUI	Nessuna logica, nessun import da src/
environment.py	grid, _objects_truth (privato), stigma_map, occupancy, traffic_log, spawn_queue, is_walkable() con sensi unici, update_stigma()	Logica di movimento degli agenti
communication.py	Costanti performative, build_payload(), parse_payload(). Solo struttura messaggi.	Logica di scambio (va nei metodi degli agenti)

pathfinding.py	astar(), real_distance(). Il caching del percorso e' attributo dell'agente (cached_path), non di questo modulo.	Raycasting (va in sensors.py)
sensors.py	has_line_of_sight(), get_visible_objects(), detect_adjacent_agents(), detect_agents_in_radius()	Calcolo percorsi A* (va in pathfinding.py)
base_agent.py	Stati interni, batteria, priorita', decide_action() astratto, merge_knowledge() con flag OGGETTO_PRESO	Strategie specifiche dei singoli agenti
logger.py	Lista eventi in RAM, dump JSON unico a fine run (struttura ispirata all'esempio del README del prof)	Write su disco ad ogni tick
renderer.py	Tutto matplotlib. Eredita la logica di visualize_environment.py + aggiunge agenti live + overlay stigmergia. Attivo solo se GUI=True.	Logica di simulazione

3 Roadmap di Sviluppo

7 fasi sequenziali — non passare alla fase successiva senza test verificato

Ogni fase produce un componente testabile in isolamento. Il criterio di avanzamento e' semplice: scrivi un assert, eseguilo, se passa procedi.

FASE 0 — Setup Iniziale e Analisi Visiva della Mappa

Obiettivo: ambiente configurato, mappe visualizzate, struttura cartelle creata.

- Crea la struttura cartelle esatta della Sezione 2.
- Copia i file del professore: A.json, B.json in data/; visualize_environment.py in tools/.
- Crea requirements.txt con: matplotlib, numpy, heapq (stdlib).
- Crea src/config.py con il contenuto completo della Sezione 1.
- Genera le immagini: python tools/visualize_environment.py data/A.json data/A.png
- Analisi visiva obbligatoria: conta i corridoi a 1 cella, individua i 4 ENTRANCE, stima quali saranno i colli di bottiglia naturali. Annota le osservazioni — serviranno per interpretare le heatmap finali.

FASE 1 — Environment: Griglia, Oggetti e Sensi Unici

Obiettivo: caricare il JSON, gestire gli oggetti come ground truth, implementare is_walkable() con sensi unici.

1.1 Classe Environment in src/environment.py

- Metodo load(path): legge JSON, popola grid, warehouses, _objects_truth, n, stigma_map, occupancy, traffic_log, delivered, spawn_queue.
- _objects_truth: set privato di tuple (r,c). NON esporre direttamente agli agenti.
- is_object_at(r,c): chiamato SOLO dal modulo sensors.py per simulare la percezione visiva.
- deliver_object(r,c): rimuove da _objects_truth, incrementa self.delivered.

1.2 is_walkable(r, c, agent_r, agent_c) — sensi unici

La firma include la posizione corrente dell'agente per determinare da quale direzione si sta avvicinando:

```
def is_walkable(self, r, c, agent_r=None, agent_c=None):
    if r < 0 or r >= self.n or c < 0 or c >= self.n:
        return False
    val = self.grid[r][c]
    if val == WALL:
```

```
        return False
    if val == ENTRANCE and agent_r is not None:
        # trova a quale magazzino appartiene questa entrance
        wh = self._warehouse_of(r, c, 'entrance')
        if wh is None: return True
        return self._coming_from_outside(r, c, agent_r, agent_c, wh['side'])
    if val == EXIT and agent_r is not None:
        wh = self._warehouse_of(r, c, 'exit')
        if wh is None: return True
        return self._coming_from_inside(r, c, agent_r, agent_c, wh['side'])
    return True  # EMPTY, WAREHOUSE

def _coming_from_outside(self, r, c, ar, ac, side):
    if side == 'top':    return ar < r   # agente a nord della entrance
    if side == 'bottom': return ar > r
    if side == 'left':   return ac < c
    if side == 'right':  return ac > c
    return True

def _coming_from_inside(self, r, c, ar, ac, side):
    # esce in direzione opposta rispetto all'entrata
    if side == 'top':    return ar > r
    if side == 'bottom': return ar < r
    if side == 'left':   return ac > c
    if side == 'right':  return ac < c
    return True
```

1.3 update_stigma() e traffic_log

- update_stigma(): ogni tick applica stigma_map[r][c] *= (1 - EVAPORATION_RATE). Azzera valori sotto 0.01. [Checklist #11]
- log_traffic(r,c): chiamato ogni volta che un agente calpesta una cella ENTRANCE o EXIT. Incrementa traffic_log[(r,c)]. Usato da analyze_results.py per le heatmap.

1.4 try_spawn_next(active_agents) [Checklist #8]

- Se spawn_queue non e' vuota E (0,0) non e' in occupancy: estrai il primo agente, imposta pos=(0,0) e state='DEPLOY', aggiungilo ad active_agents.

1.5 Test di verifica Fase 1

```
env = Environment(); env.load('data/A.json')
assert len(env.grid) == 25 and len(env.grid[0]) == 25
assert len(env.warehouses) == 4
assert env.delivered == 0
# test senso unico: trova una entrance 'top' e simula agente che arriva da nord
wh = env.warehouses[0]
er, ec = wh['entrance']
assert env.is_walkable(er, ec, er-1, ec) == True   # arriva da nord -> OK
assert env.is_walkable(er, ec, er+1, ec) == False  # arriva da sud -> BLOCCATO
print('Fase 1 OK')
```

FASE 2 — Pathfinding e Sensori

Obiettivo: A* funzionante che rispetta is_walkable(), line-of-sight con occlusione, path caching.

2.1 astar() in src/pathfinding.py

- Firma: astar(env, start, goal, stigma_map=None). Usa env.is_walkable() per ogni cella esplorata, passando come agent_r/agent_c la posizione del nodo corrente nell'albero di ricerca.
- Heuristic = distanza Manhattan. Se stigma_map: costo cella += stigma_map[r][c].
- Ritorna lista di tuple (r,c) da start a goal inclusi. Ritorna [] se nessun percorso esiste.
- real_distance(env, start, goal): ritorna len(astar(...))-1. NON distanza Manhattan. Usata da Worker 3 per la valutazione energetica. [Checklist #1]

2.2 has_line_of_sight() in src/sensors.py

- Raycasting lineare con risoluzione ambiguita' diagonale: se il percorso ha uno step diagonale, controlla entrambi i cammini intermedi (via r poi c, oppure via c poi r). La cella e' visibile se almeno uno dei due percorsi e' libero da muri. [Checklist #3]
- get_visible_objects(env, r, c, radius): per ogni cella nel rombo Manhattan di raggio 'radius', se env.is_object_at(nr,nc) e has_line_of_sight() ritorna True, aggiunge (nr,nc) alla lista.
- detect_adjacent_agents(occupancy, r, c): controlla le 4 celle adiacenti. NON usa raycasting. Usato in DEPLOY e risoluzione collisioni.
- detect_agents_in_radius(occupancy, r, c, radius): conta agenti nel raggio. Senza occlusione. Usato da Worker 2 per malus congestione.

2.3 Test di verifica Fase 2

```
env = Environment(); env.load('data/A.json')
path = astar(env, (0,0), (5,5))
assert len(path) > 0 and path[0]==(0,0) and path[-1]==(5,5)
# ogni passo deve essere is_walkable
for i in range(len(path)-1):
    pr,pc = path[i]; nr,nc = path[i+1]
    assert env.is_walkable(nr, nc, pr, pc), f'passo non walkable: {path[i]}-
>{path[i+1]}'
dist = real_distance(env, (0,0), (10,10))
assert dist >= 20
print('Fase 2 OK')
```

FASE 3 — BaseAgent e Ciclo Principale

Obiettivo: BaseAgent con ciclo di vita, gestione batteria, merge_knowledge. Loop principale in main.py.

3.1 Classe BaseAgent in src/agents/base_agent.py

- Attributi: id, state, pos=(r,c), battery=BATTERY_INITIAL, carrying=False, carrying_obj=None, local_map, cached_path=[].
- Stati: DEPLOY, EXPLORE, MOVE_TO_OBJECT, PICK_UP, RETURN_TO_WAREHOUSE, DELIVER, LOW_BATTERY, RETURN_SAFE, STANDBY.
- check_battery(env): se battery < real_distance(env, pos, nearest_entrance)*ENERGY_MARGIN: commuta in LOW_BATTERY. [Checklist #1]
- nearest_entrance(env): itera env.warehouses, calcola real_distance() a ogni entrance, ritorna la (r,c) con distanza minima. [Checklist #4]
- decide_action(env, tick): metodo astratto. Ogni sottoclasse lo sovrascrive.

3.2 merge_knowledge() [Checklist #9]

```
def merge_knowledge(self, incoming_map, tick):
    for cell, data in incoming_map.items():
        existing = self.local_map.get(cell)
        if existing and existing['status'] == 'TAKEN':  # priorita' assoluta
            continue
        if existing is None or data['ts'] > existing['ts']:
            self.local_map[cell] = data

def mark_taken(self, r, c):  # chiamato quando l'agente raccoglie un oggetto
    self.local_map[(r,c)] = { 'status':'TAKEN', 'ts': float('inf') }

def mark_abandoned(self, r, c, tick):  # chiamato in LOW_BATTERY con carico
    self.local_map[(r,c)] = { 'status':'ABANDONED', 'ts': tick }
```

3.3 Loop principale in main.py [Checklist #7, #12]

```
import random
from src.config import *
from src.environment import Environment
from src.utils.logger import Logger
from src.utils.renderer import Renderer

env = Environment()
env.load(f'data/{INSTANCE}.json')   # INSTANCE = 'A' o 'B'
logger = Logger()
renderer = Renderer(env) if GUI else None

active_agents = []
tick = 0
while tick < MAX_TICKS:
    env.try_spawn_next(active_agents)         # spawn queue
    random.shuffle(active_agents)              # asincronia: shuffle ogni round
    for agent in active_agents:
        agent.decide_action(env, tick)         # 1 tick = 1 mossa di 1 agente
        agent.battery -= 1
        logger.log(tick, agent)
    env.update_stigma()                        # evaporazione feromone
    if renderer: renderer.draw(active_agents, tick)
    if env.delivered == NUM_OBJECTS:            # early stopping
        print(f'Completato al tick {tick}')
```

```
        break
    tick += 1

logger.dump(f'outputs/logs/{INSTANCE}_seed{SEED}.json')
```

FASE 4 — Implementazione Agenti (Ordine Consigliato)

Ordine: Worker1 -> Scout2 -> Worker2 -> Worker3 -> Scout1. Testa ogni agente singolarmente prima del successivo.

4.1 Worker 1 — Greedy (src/agents/workers.py)

- EXPLORE: se nessun target noto in local_map, vai verso la cella UNKNOWN piu' vicina. Usa detect_adjacent_agents() per sapere se sei isolato dal DEPLOY.
- MOVE_TO_OBJECT: chiama astar() UNA volta, salva in self.cached_path. Ogni tick pop(0) dalla lista. Se la prossima cella e' occupata o non piu' walkable: ricalcola A*. [Checklist #14]
- RETURN_TO_WAREHOUSE: target = nearest_entrance(env). Usa astar() verso quella cella specifica. [Checklist #4]
- DELIVER: quando calpesta ENTRANCE, chiama env.log_traffic(r,c), muovi un passo dentro (WAREHOUSE), chiama env.deliver_object(), chiama mark_taken(). Imposta target = exit di quel magazzino. [Checklist #5]
- Dopo EXIT: chiama env.log_traffic(r,c) sull'exit, commuta in EXPLORE.

4.2 Scout 2 — Esploratore Stigmergico (src/agents/scouts.py)

- Vettore probabilita' [N,S,E,O] = [0.25]*4. Per ogni direzione verso cella in local_map con status VISITED: prob *= 0.30 (penalita' 70%).
- Normalizza e campiona con random.choices(). Se tutte le direzioni sono muri o fuori griglia: STANDBY 1 tick.
- Ogni volta che e' in COMM_RADIUS con un altro agente: build_payload() e merge_knowledge(). Propaga anche eventuali ABANDONED in local_map. [Checklist #10]

4.3 Worker 2 — Sensibile alla Densita'

- Identico a Worker 1, ma prima di ogni passo: count = detect_agents_in_radius(env.occupancy, r, c, COMM_RADIUS). Se count >= 2: stigma_map[nr][nc] += CONGESTION_MALUS per le celle della direzione pianificata. Ricalcola A* con stigma aggiornata.
- Il ricalcolo e' giustificato: il malus ha modificato il grafo, il cached_path non e' piu' ottimale.

4.4 Worker 3 — Sweeper Risk-Averse

- Prima di commutare in MOVE_TO_OBJECT: d_obj = real_distance(env, pos, obj_pos); d_wh = real_distance(env, obj_pos, nearest_entrance(env)). Entrambe via pathfinding. [Checklist #1]
- Se battery > (d_obj + d_wh) * ENERGY_MARGIN: commuta. Altrimenti: ignora il task, continua esplorazione sistematica della frontiera (celle UNKNOWN adiacenti a celle gia' esplorate).

- LOW_BATTERY con oggetto in carico: deposita l'oggetto, chiama mark_abandoned(r,c,tick), propaga ABANDONED al prossimo contatto FIPA-ACL. [Checklist #10]

4.5 Scout 1 — Wall-Follower Reattivo

- Left-Hand Rule: tieni il muro alla sinistra. Priorita' direzioni: front-of-wall -> parallela -> opposta.
- self.stress += 1 ogni passo lungo muro. Se stress > STRESS_MAX e nessuna nuova cella scoperta: STRESS_RANDOM_STEPS mosse verso UNKNOWN, azzera stress.
- Usa get_visible_objects() ad ogni tick per rilevare oggetti nel raggio VISION_RADIUS con occlusione.

FASE 5 — Comunicazione FIPA-ACL

5.1 src/communication.py — solo struttura

ATTENZIONE dipendenze circolari: communication.py definisce SOLO struttura messaggi. La logica di scambio va nei metodi degli agenti. NON importare agents/ da qui.

```
INFORM    = 'INFORM'
TAKEN     = 'TAKEN'
ABANDONED = 'ABANDONED'
VISITED   = 'VISITED'
UNKNOWN   = 'UNKNOWN'

def build_payload(sender_id, local_map, tick):
    return { 'perf': INFORM, 'sender': sender_id,
             'ts': tick, 'map': local_map }

def parse_payload(msg):
    return msg.get('map', {})
```

FASE 6 — Logger e Renderer (Ispirato ai File del Professore)

6.1 src/utils/logger.py — struttura ispirata al README del prof

Il README del professore mostra esattamente la struttura di log desiderata. Il tuo logger deve produrre output compatibile:

```
class Logger:
    def __init__(self): self.events = []

    def log(self, tick, agent):
        self.events.append({
            'tick': tick,
            'id':   agent.id,
            'pos':  list(agent.pos),
            'battery': agent.battery,
            'carrying': agent.carrying,
```

```
            'state': agent.state,
        })

    def dump(self, path):
        # scrive tutto in UNO shot a fine simulazione, NON ad ogni tick
        metrics = self._compute_metrics()
        with open(path, 'w') as f:
            json.dump({'metrics': metrics, 'events': self.events}, f, indent=2)

    def _compute_metrics(self):
        # tick completamento, energia media, fallimenti critici
        ...
```

6.2 src/utils/renderer.py — eredita da visualize_environment.py

Copia la logica di disegno da visualize_environment.py e aggiungici il layer degli agenti e della stigmergia:

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

class Renderer:
    def __init__(self, env):
        self.env = env
        self.fig, self.ax = plt.subplots(figsize=(10,10))
        plt.ion()
        self._draw_static_grid()   # chiama la logica di visualize_environment.py

    def draw(self, agents, tick):
        self.ax.clear()
        self._draw_static_grid()
        self._draw_stigma_overlay()  # heatmap semitrasparente sulla stigmergia
        for agent in agents:
            r, c = agent.pos
            # IMPORTANTE: matplotlib usa (x=colonna, y=riga), come nel prof
            self.ax.plot(c, r, 'o', markersize=10, label=f'A{agent.id}')
        self.ax.invert_yaxis()  # riga 0 in alto, come in visualize_environment.py
        self.ax.set_title(f'Tick {tick} | Consegnati: {self.env.delivered}')
        plt.pause(0.05)
```

4 Checklist Master — 16 Punti

Da spuntare durante la stesura del codice, fase per fase

	Fase	Task	File	Priorita'
☐	**PREREQUISITO**	#13 — Usa (r,c) ovunque. MAI (x,y). In matplotlib ricorda: x=col, y=riga.	Tutti	**DAL PRIMO RIGO**
☐	**PREREQUISITO**	#15 — random.seed(42) e np.random.seed(42) come prime righe.	config.py	**DAL PRIMO RIGO**
☐	**PREREQUISITO**	#16 — GUI=True/False in config.py. renderer.py usa importazione condizionale.	config.py	**DAL PRIMO RIGO**
☐	**FASE 0**	Genera A.png e B.png. Studia visivamente la mappa prima di scrivere codice.	tools/	Prima di tutto
☐	**FASE 1**	#2 — is_walkable() usa 'side' da warehouses per sensi unici ENTRANCE/EXIT.	environment.py	Test unitario
☐	**FASE 1**	objects e' ground truth privato (_objects_truth). Agenti non lo leggono mai.	environment.py	Architetturale
☐	**FASE 1**	#11 — update_stigma() con evaporazione. traffic_log su ogni ENTRANCE/EXIT.	environment.py	Test unitario
☐	**FASE 1**	#8 — Spawn queue: agente N+1 entra solo quando (0,0) e' libera.	environment.py	Test unitario
☐	**FASE 2**	#1 — real_distance() via pathfinding. Entrambe le distanze in Worker 3.	pathfinding.py	Critico W3
☐	**FASE 2**	#4 — astar() passa agent_r/agent_c a is_walkable() in ogni nodo.	pathfinding.py	Test con mappa
☐	**FASE 2**	#3 — Raycasting solo per oggetti a raggio 1-3. Raggio 1 = controllo diretto.	sensors.py	Ottimizzazione
☐	**FASE 3**	#7 — random.shuffle(agents). 1 tick = 1 mossa di 1 agente.	main.py	Loop principale
☐	**FASE 3**	#12 — Early stopping: break quando env.delivered == NUM_OBJECTS.	main.py	Loop principale
☐	**FASE 4**	#14 — cached_path negli agenti deliberativi. Ricalcola solo se bloccato.	workers.py	W1, W2, W3
☐	**FASE 4**	#5 — DELIVER: log_traffic(ENTRANCE), step dentro, deliver, log_traffic(EXIT).	workers.py	Tutti Worker
☐	**FASE 5**	#9 — STATUS TAKEN con ts=inf. Non sovrascrivibile mai in merge_knowledge.	base_agent.py	merge_knowledge
☐	**FASE 5**	#10 — ABANDONED propagato via FIPA-ACL al prossimo contatto.	workers.py	Worker 3

5 Edge Cases Critici

Situazioni limite da testare esplicitamente — non aspettare le run statistiche

Agente tenta di entrare da EXIT o uscire da ENTRANCE	is_walkable() deve bloccare il movimento e ritornare False. L'agente deve restare fermo (STANDBY 1 tick) e cercare il percorso corretto. Senza questo controllo, gli agenti attraversano i magazzini in direzione sbagliata.
A* ritorna [] — percorso non esiste	L'agente va in STANDBY per 3 tick, poi riprova. Non deve crashare. Logger: PATH_NOT_FOUND con posizione e tick. Puo' succedere se un agente e' intrappolato da altri agenti che bloccano tutti i corridoi.
Worker in LOW_BATTERY con oggetto in carico	Deposita oggetto nella cella corrente (ABANDONED). NON tenta di consegnare. Chiama mark_abandoned() e propaga via FIPA-ACL. Poi RETURN_SAFE verso nearest_entrance.
Due Worker puntano allo stesso oggetto	Il primo che arriva raccoglie e chiama mark_taken() (ts=inf). Il secondo, tramite merge_knowledge(), trovera' status=TAKEN e tornera' in EXPLORE senza compiere il viaggio a vuoto. [Checklist #9]
Corridoio 1 cella: stallo frontale	Regola deterministica: Carico > Scarico > ID minore. L'agente a bassa priorita' fa backtracking (o STANDBY se nessuno spazio laterale). [Checklist #6]
ax.invert_yaxis() mancante nel renderer	La visualizzazione live mostra la mappa capovolta rispetto a A.png / B.png. Tutti gli agenti sembrano andare nella direzione sbagliata. Verifica sempre che il renderer usi invert_yaxis() come visualize_environment.py.
Batteria = 0 lontano dal magazzino	Fallimento critico. Logger: BATTERY_DEAD con posizione e tick. Agente diventa inattivo. Contato nelle metriche come 'fallimenti critici'. Prevenuto dal check_battery() ad ogni tick.

6 Configurazioni Sperimentali

B0, C1, C2, C3 — come modificare config.py e quali metriche raccogliere

Per ogni configurazione, modifica SOLO i parametri indicati. Questo garantisce che le differenze misurate siano causate unicamente dalla variabile manipolata e non da altri fattori.

Config	Variabile manipolata	Come impostare	Cosa misurare
B0	Tutti Random Walk, no stigma, no ACL	Tutti gli agenti: decide_action() = random walk puro. STIGMA_ON=False. Nessun build_payload().	Lower bound su tutte le metriche. Heatmap congestione massima attesa.
C1	VISION_RADIUS = 1 oppure 3	2 set di run separate. Prima VISION_RADIUS=1 (5 run), poi =3 (5 run). Stessa architettura agenti.	% mappa esplorata a tick 100/250/500. Raggio stretto = piu' congestione agli ingressi?
C2	STIGMA_ON = True oppure False	Con STIGMA_ON=False: Worker 2 usa A* senza malus congestione, diventa identico a Worker 1.	Distribuzione traffico tra ENTRANCE. Stigma distribuisce uniformemente il flusso?
C3	INSTANCE = 'A' oppure 'B'	Stessa configurazione completa su A.json e B.json. Confronta le heatmap delle due mappe.	Quali ENTRANCE diventano colli di bottiglia? La topologia determina l'asimmetria?

Procedura N=5 run per ogni configurazione

1. Imposta la configurazione in config.py (es. VISION_RADIUS=1).
2. In run_configs.py itera: SEEDS = [42, 123, 456, 789, 1337].
3. Per ogni seed: random.seed(s), np.random.seed(s), GUI=False, lancia simulazione.
4. logger.dump() salva in outputs/logs/CONFIG_INSTANCE_SEED.json.
5. analyze_results.py legge i 5 JSON, calcola media +- std su tutte le metriche.
6. Genera heatmap aggregata sommando i traffic_log delle 5 run.

Metriche da raccogliere per ogni run (da sezione 5.5 della proposta)

- Oggetti consegnati al termine dei tick massimi.
- Tick totali per completamento (grazie all'early stopping e' il tick esatto).
- Energia media consumata per agente = (BATTERY_INITIAL - battery_finale) / NUM_AGENTS.
- Fallimenti critici (BATTERY_DEAD): numero di agenti rimasti a 0 lontano dal magazzino.
- Percentuale mappa esplorata a Tick 100, 250, 500: conta celle con status != UNKNOWN in local_map unione di tutti gli agenti.
- Heatmap congestione: traffic_log aggregato su N=5 run. Visualizzata con draw_heatmap() in renderer.py.
- Flusso per ENTRANCE: per ognuno dei 4 ingressi, conta i transiti totali — identifica il collo di bottiglia primario.